

Оглавление

1.	Введение	3
2.	Основная часть	3
2.1.	Анализ данных	3
3.	Обучение моделей	6
3.1.	Линейная регрессия	7
3.2.	Полиномиальная регрессия со стохастическим градиентным спуском	9
3.3.	Дерево решений	12
3.4.	Случайный лес	14
3.5.	Метод опорных векторов	15
3.6.	Гребневая регрессия	16
4.	Заключение	17
5.	Список источников	18
6.	Приложение	18

1. Введение

Целью научно-исследовательской работы является исследование задачи регрессии с применением разных подходов: градиентный спуск, регуляризацию, случайный лес, SVM, оценка их эффективности и времени работы. Возьмем набор данных с информацией о домах и будем предсказывать их цену, опираясь на остальные параметры, как площадь, количество комнат, местоположение и так далее.

2. Основная часть

Работать будем с языком программирования Python, для реализации машинного обучения воспользуемся известной библиотекой Scikit-learn, также для вспомогательных целей нам потребуются библиотеки Pandas и NumPy.

2.1. Анализ данных

Реальные данные без предварительной обработки скорее всего не будут подходить для алгоритмов машинного обучения. Так как могут быть недостающие значения, текстовые атрибуты, ненужные параметры, выбросы, поэтому перед обучением моделей возможно потребуется провести некоторые процедуры для приведения данных в надлежащий вид.

Взглянем на нашу таблицу:

	area_type	availability	location	size	society	total_sqft	bath	balcony	price
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Coomee	1056	2.0	1.0	39.07
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600	5.0	3.0	120.00
2	Built-up Area	Ready To Move	Uttarahalli	3 BHK	NaN	1440	2.0	3.0	62.00
3	Super built-up Area	Ready To Move	Lingadheeranahalli	3 BHK	Soiewre	1521	3.0	1.0	95.00
4	Super built-up Area	Ready To Move	Kothanur	2 BHK	NaN	1200	2.0	1.0	51.00
...	...	...	...	...	...	...	...	...	...
13315	Built-up Area	Ready To Move	Whitefield	5 Bedroom	ArsiaEx	3453	4.0	0.0	231.00
13316	Super built-up Area	Ready To Move	Richards Town	4 BHK	NaN	3600	5.0	NaN	400.00
13317	Built-up Area	Ready To Move	Raja Rajeshwari Nagar	2 BHK	Mahla T	1141	2.0	1.0	60.00
13318	Super built-up Area	18-Jun	Padmanabhanagar	4 BHK	SollyCl	4689	4.0	1.0	488.00
13319	Super built-up Area	Ready To Move	Doddathoguru	1 BHK	NaN	550	1.0	1.0	17.00

Сразу избавимся от столбцов «availability» и «society» за ненадобностью.

```
table = table.drop("availability", axis=1)
table = table.drop("society", axis=1)
```

Заметим, что у нас есть текстовые параметры, которые необходимо преобразовать в числовые для работы алгоритмов машинного обучения. Самым простым решением было бы сопоставить каждую категорию с отличающимся целым числом, но это может негативно повлиять на точность работы, так как алгоритмы машинного обучения будут предполагать, что два соседних значения более похожи, чем два отдаленных значения, а это не всегда верно. Атрибут «area_type» принимает всего 4 значения, так что можем воспользоваться унитарным кодированием. На каждую категорию создадим один двоичный атрибут.

```
dummy_cols = pd.get_dummies(table.area_type).drop(
    'Super_built-up_Area', axis='columns')
table = pd.concat([table, dummy_cols], axis='columns')
table = table.drop("area_type", axis=1)
```

А вот параметр «site_location» принимает 98 значений, поэтому придется просто каждому значению биективно сопоставить какое-то целое число, после этого снизится точность, но иначе время работы алгоритмов сильно возрастет.

```
table["site_location"].value_counts()
table["site_location"] = table["site_location"].astype(
    'category')
table.dtypes
table["site_location"] = table["site_location"].cat.codes
```

Параметр «size» помимо количества комнат в доме содержит лишнюю текстовую информацию, исправим это.

Обработаем атрибут «size»:

```
table['num_size'] = table['size'].apply(
    lambda x: int(x.split('_')[0]))
table = table.drop("size", axis=1)
```

Разберемся с параметром «total_sqft», иногда вместо числа он принимает значения наподобие «2Acres», «300Sq.Yards» избавимся от таких округов, также попадают диа-

пазоны, например «3369-3464» в таких случаях будем брать среднее значение:

```
def sqft_fix(x):
    t = x.split('-')
    if len(t) == 2:
        return(float(t[0]) + float(t[1]))/2
    try:
        return float(x)
    except:
        return None

table["total_sqft"] = table["total_sqft"].apply(sqft_fix)
```

К тому же в обучающих данных полно ошибок, выбросов, шумов, так что надо необходимо провести очистку. После рассмотрим матрицу корреляции.

```
table.corr()["price"].sort_values(ascending=False)
price                1.000000
total_sqft           0.921517
bath                 0.641973
num_size             0.610434
Plot Area            0.191125
balcony              0.118014
Built-up Area        0.017445
Carpet Area          -0.012209
site_location        -0.015900
Name: price, dtype: float64
```

Получается наибольшим влиянием обладают атрибуты «total_sqft», «bath», «num_size». За немногими исключениями алгоритмы машинного обучения не особенно хорошо выполняются, когда входные числовые атрибуты имеют очень разные масштабы. Воспользуемся масштабирование по минимаксу. Масштабирование по минимаксу выполняется довольно просто: значения смещаются и изменяются так, чтобы в итоге находиться в диапазоне от 0 до 1. Это делается путем вычитания минимального значения и деления на разность максимального и минимального значений. Для решения такой задачи в Scikit-Learn предназначен трансформатор по имени MinMaxScaler.

```

table_col = table.columns.values
MMS = MinMaxScaler(copy=False).fit(table)
table = MMS.transform(table)
table = pd.DataFrame(table, columns=table_col)

```

Разобьем набор данных на обучающий и тестовый.

```

table_labels = table['price'].copy()
table_ready = table.drop('price', axis='columns')
train_set, test_set, train_labels, test_labels =
train_test_split(
table_ready, table_labels, test_size = 0.2, random_state = 42)

```

Теперь наши данные готовы к использованию алгоритмов машинного обучения.

3. Обучение моделей

Начнем обучение регрессионных моделей, но перед этим разберемся со способами проверки. Точность будем проверять вручную на обучающих и тренировочных наборах, также используем метод кросс-валидации (методика обучения и оценки модели, которая разбивает данные на несколько секций и обучает несколько алгоритмов на этих секциях). В качестве основного критерия качества будем использовать квадратный корень из среднеквадратической ошибки (RMSE), он дает представление о том, насколько большую ошибку система обычно допускает в своих прогнозах, с более высоким весом для крупных ошибок.

Математическая формула для вычисления RMSE:

$$\text{RMSE}(X, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2}$$

Но в особых случаях нам может понадобиться средняя абсолютная ошибка (MAE).

Математическая формула для вычисления MAE:

$$\text{MAE}(X, h) = \frac{1}{m} \sum_{i=1}^m |h(x^{(i)}) - y^{(i)}|$$

m - количество образцов в наборе данных, на которых измеряется ошибка.

$x^{(i)}$ - вектор всех значений признаков (исключая метку) i -го образца в наборе данных, а $y^{(i)}$ - его метка (желательное выходное значение для данного образца).

X - матрица, содержащая все значения признаков (исключая метки) всех образцов в наборе данных.

h - функция прогнозирования системы.

Также рассмотрим кривые обучения - график изменения величины ошибки в процессе обучения модели.

Функция построения кривой обучения:

```
def plot_learning_curves(model):
    X_train = train_set.values
    X_val = test_set.values
    y_train = train_labels.values
    y_val = test_labels.values
    train_errors, val_errors = [], []
    for m in range(1, len(X_train)):
        model.fit(X_train[:m], y_train[:m])
        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)
        train_errors.append(mean_squared_error(
            y_train_predict, y_train[:m]))
        val_errors.append(mean_squared_error(
            y_val_predict, y_val))
    plt.plot(np.sqrt(train_errors), "r--", linewidth=1,
             label="train")
    plt.plot(np.sqrt(val_errors), "b-", linewidth=3,
             label="val")
    plt.legend(loc="upper right", fontsize=14)
    plt.xlabel("Training set size", fontsize=14)
    plt.ylabel("RMSE", fontsize=14)
```

3.1. Линейная регрессия

Линейная регрессия - это модель зависимости переменной x от одной или нескольких других переменных с линейной функцией зависимости.. Как правило, линейная модель вырабатывает прогноз, просто вычисляя взвешенную сумму входных признаков

плюс константы под названием член смещения, также называемой свободным членом, как показано в уравнении.

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n.$$

y - спрогнозированное значение

n - количество признаков

x_i - значение i -того признака

θ_j - j -тый параметр модели (включая член смещения θ_0 и веса признаков $\theta_1, \theta_2, \dots, \theta_n$).

Хорошо, мы имеем линейную регрессионную модель, но как мы будем ее обучать? Обучение модели означает установку ее параметров так, чтобы модель была наилучшим образом подогнана к обучающему набору. Для этой цели нам первым делом нужна мера того, насколько хорошо модель подогнана к обучающим данным. Вспомним про RMSE. Следовательно, для обучения линейной регрессионной модели необходимо найти значение θ , которое сводит к минимуму RMSE. На деле проще довести до минимума среднеквадратическую ошибку (MSE), что приведет к тому же самому результату, поскольку значение, которое сводит к минимуму функцию, также сводит к минимуму ее квадратный корень.

Функция издержек MSE для линейной регрессионной модели

$$\text{MSE}(X, h) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$$

Для нахождения значения θ , которое сводит к минимуму функцию издержек, имеется решение в аналитическом виде - иными словами, математическое уравнение, дающее результат напрямую. Оно называется нормальным.

Нормальное уравнение

$$\theta_{best} = (X^T * X)^{-1} * X^T * y$$

θ_{best} - значение θ , которое сводит к минимуму функцию издержек.

y - вектор целевых значений от $y^{(1)}$ до $y^{(m)}$.

Теперь воспользуемся реализацией алгоритма из библиотеки Scikit-Learn, и применим к нашим данным. Вычислительная сложность приблизительно $O(n^3)$.

Результаты алгоритма:

Train set:

Predictions: [0.02325407 0.02773283 0.02208574 0.02705058 0.040113]
 Labels: [0.016267942583732056, 0.02727272727272727, 0.022966507177033493, 0.015789473684210527, 0.040669856459330143]
 RMSE: 0.0135558644379661

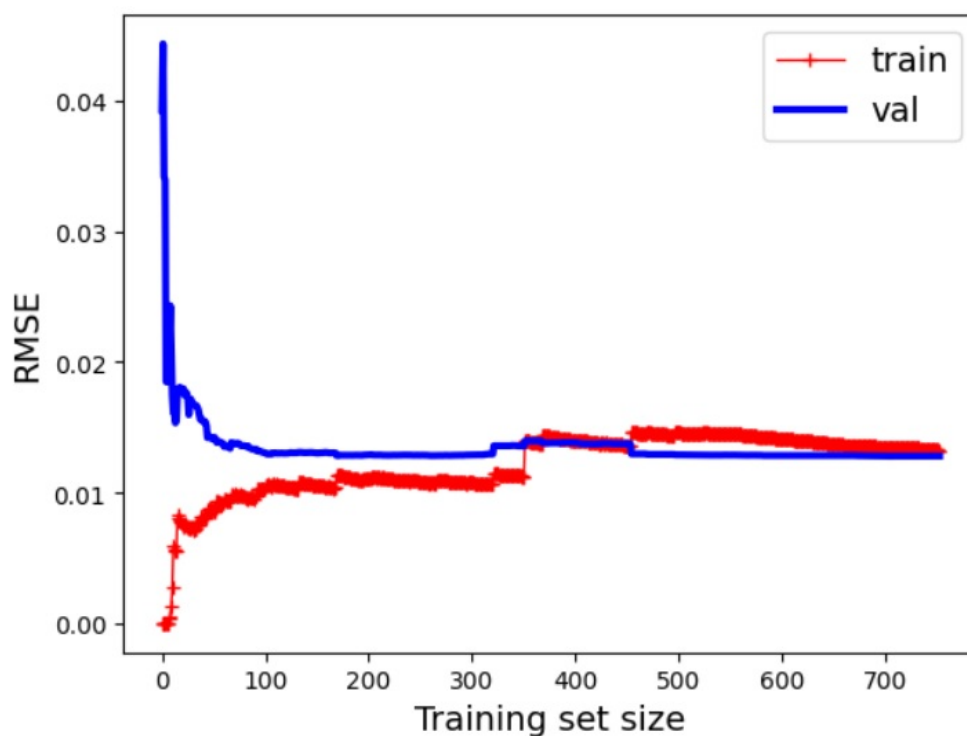
Test set:

Predictions: [0.01718381 0.03004695 0.03244494 0.03536181 0.02820693]
 Labels: [0.008133971291866028, 0.03110047846889952, 0.02631578947368421, 0.029665071770334926, 0.023669856459330143]
 RMSE: 0.012791044924997233

Cross Validation:

Scores: [0.01423597 0.01334382 0.01634227 0.01238261 0.01168626 0.01289523
 0.01468245 0.01353159 0.01528776 0.01295365]
 Average: 0.013734160146394734
 Standard deviation: 0.001337682160169626

Кривая обучения:



Заметим, что ошибка модели не так плоха: 0.0137, кривые сходятся.

3.2. Полиномиальная регрессия со стохастическим градиентным спуском

Градиентный спуск представляет собой самый общий алгоритм оптимизации, способный находить оптимальные решения широкого диапазона задач. Основная идея градиентного спуска заключается в том, чтобы итеративно подстраивать параметры для сведения к минимуму функции издержек. Начнем с заполнения вектора θ случайными значениями. Затем мы постепенно улучшаем его, предпринимая по одному маленькому шагу за раз и на каждом шаге пытаемся снизить функцию издержек до тех пор, пока

алгоритм не сойдется в минимуме. Важным параметром в градиентном спуске является размер шагов, определяемый гиперпараметром скорости обучения. Если скорость обучения слишком мала, тогда алгоритму для сведения придется пройти через множество итераций, что потребует длительного времени. С другой стороны, если скорость обучения слишком высока, тогда вы можете перескочить долину и оказаться на другой стороне, возможно даже выше, чем находились ранее. Это способно сделать алгоритм расходящимся, что приведет к выдаче постоянно увеличивающихся значений и неудаче в поиске хорошего решения. Наконец, не все функции издержек выглядят как точные правильные чаши. Могут существовать впадины, выступы, плато и самые разнообразные участки нерегулярной формы, которые крайне затрудняют сходжение к минимуму. К счастью, функция издержек MSE для линейной регрессионной модели является выпуклой функцией. Отсюда следует, что локальные минимумы отсутствуют, а есть только один глобальный минимум. Она также представляет собой непрерывную функцию с наклоном, который никогда не изменяется неожиданным образом. Упомянутые два факта имеют большое значение: градиентный спуск гарантированно подберется произвольно близко к глобальному минимуму. Чтобы реализовать градиентный спуск, вам понадобится вычислить градиент функции издержек в отношении каждого параметра модели θ_j . Другими словами, вам необходимо подсчитать, насколько сильно функция издержек будет изменяться при небольшом изменении θ_j . Результат называется частной производной.

Частные производные функции издержек:

$$\frac{\delta}{\delta \theta_j} \text{MSE}(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta^T * x^{(i)} - y^{(i)}) x_j^{(i)}$$

Вместо вычисления таких частных производных по отдельности вы можно вычислить их все сразу. Вектор градиент, известный как $\Delta_{\theta} \text{MSE}(\theta)$, содержит все частные производные функции издержек по одной для каждого параметра модели.

Вектор-градиент функции издержек:

$$_{\theta} \text{MSE}(\theta) = \frac{2}{m} X^T * (X * \theta - y)$$

Имея вектор-градиент, который указывает вверх, просто двигаемся в противоположном направлении вниз. Это означает вычитание $\Delta_{\theta} \text{MSE}(\theta)$ из θ . Именно здесь в игру вступает скорость обучения η : умножение вектора градиента на η дает размер шага вниз.

Шаг градиентного спуска

$$\theta_{\text{(следующий шаг)}} = \theta - \eta \Delta_{\theta} \text{MSE}(\theta)$$

Главная проблема с пакетным градиентным спуском - тот факт, что он использует полный обучающий набор для вычисления градиентов на каждом шаге, который делает его очень медленным в случае крупного обучающего набора. Как противоположная крайность, стохастический градиентный спуск на каждом шаге просто выбирает из обучающего набора случайный образец и вычисляет градиенты на основе только этого единственного образца. Очевидно, алгоритм становится гораздо быстрее, т.к. на каждой операции ему приходится манипулировать совсем малым объемом данных. Также появляется возможность проводить обучение на гигантских обучающих наборах, потому что на каждой итерации в памяти должен находиться только один образец. С другой стороны, из-за своей стохастической природы этот алгоритм гораздо менее нормален, чем пакетный градиентный спуск: вместо умеренного понижения вплоть до достижения минимума функция издержек будет скачками изменяться вверх и вниз, понижаясь только в среднем. Со временем алгоритм будет очень близок к минимуму, но как только он туда доберется, скачкообразные изменения продолжатся, никогда не уходясь. Таким образом, после окончания алгоритма финальные значения параметров оказываются хорошими, но не оптимальными. Следовательно, случайность хороша для избегания локальных оптимумов, но плоха потому, что означает, что алгоритм может никогда не осесть на минимуме. Одним из решений такой дилеммы является постепенное сокращение скорости обучения. Шаги начинаются с больших, а затем становятся все меньше и меньше, позволяя алгоритму обосноваться на глобальном минимуме. Такой процесс называется имитацией отжига. Функция, которая определяет скорость обучения на каждой итерации, называется графиком обучения. Если скорость обучения снижается слишком быстро, то вы можете застрять в локальном минимуме или даже остаться на полпути к минимуму. Если скорость обучения снижается чересчур медленно, тогда вы можете долго прыгать возле минимума и в конечном итоге получить квазиоптимальное решение в случае прекращения обучения слишком рано. Теперь применим алгоритм.

Результаты алгоритма:

Train set:

Predictions: [0.01050009 0.02570131 0.02524466 0.02617627 0.04237463]

Labels: [0.016267942583732056, 0.02727272727272727, 0.022966507177033493, 0.015789473684210527, 0.04066985645933015]

RMSE: 0.0207340100749646

Test set:

Predictions: [0.00905577 0.02752617 0.02779998 0.02794283 0.02660376]

Labels: [0.008133971291866028, 0.03110047846889952, 0.02631578947368421, 0.029665071770334926, 0.023669856459330143]

RMSE: 0.018475587162838206

Cross Validation:

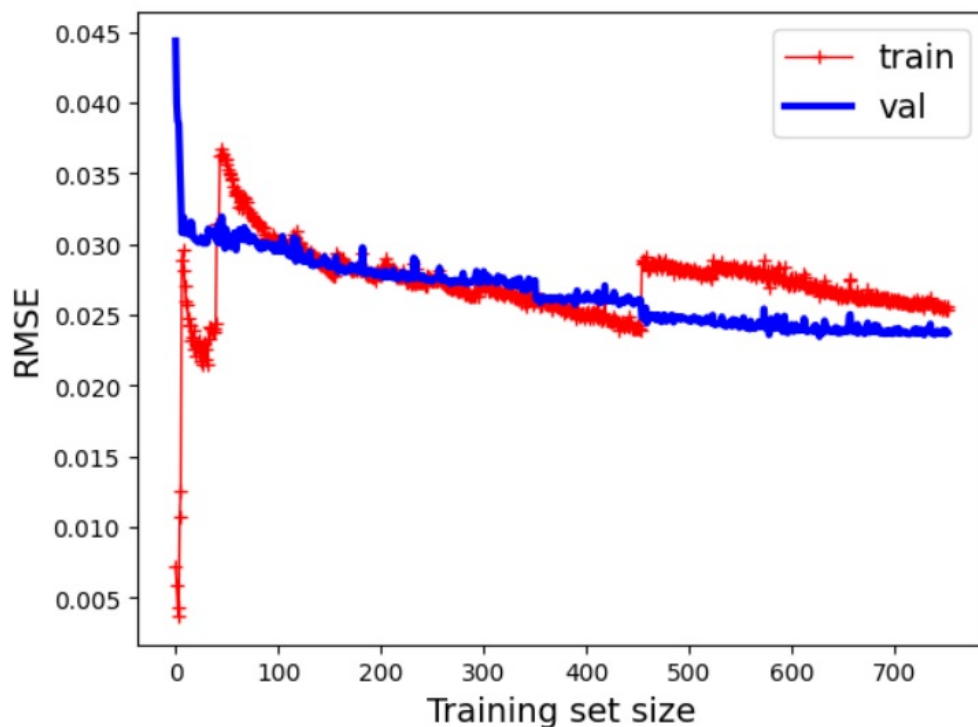
Scores: [0.02067917 0.01871545 0.02407197 0.01875224 0.01640685 0.01940554

0.02089431 0.03340856 0.02259378 0.01815743]

Average: 0.021308531150301655

Standard deviation: 0.0045508549754490054

Кривая обучения:



Ошибка немного возросла - 0.0189, но на графике можно заметить, что она медленно убывает. Возможно, что при увеличении объема данных результат улучшится.

3.3. Дерево решений

Структура дерева представляет собой «листья» и «ветки». На рёбрах («ветках») дерева решения записаны признаки, от которых зависит целевая функция, в «листьях» записаны значения целевой функции, а в остальных узлах — признаки, по которым различаются случаи. Чтобы классифицировать новый случай, надо спуститься по дереву до листа и выдать соответствующее значение. Цель состоит в том, чтобы создать

модель, которая предсказывает значение целевой переменной на основе нескольких переменных на входе. Каждый лист представляет собой значение целевой переменной, изменённой в ходе движения от корня по рёбрам дерева до листа. Каждый внутренний узел сопоставляется с одной из входных переменных.

Результаты алгоритма:

Train set:

Predictions: [0.01626794 0.02727273 0.02296651 0.01578947 0.04060048]
Labels: [0.016267942583732056, 0.02727272727272727, 0.022966507177033493, 0.015789473684210527, 0.04066985645933015]
RMSE: 0.0010716414221729502

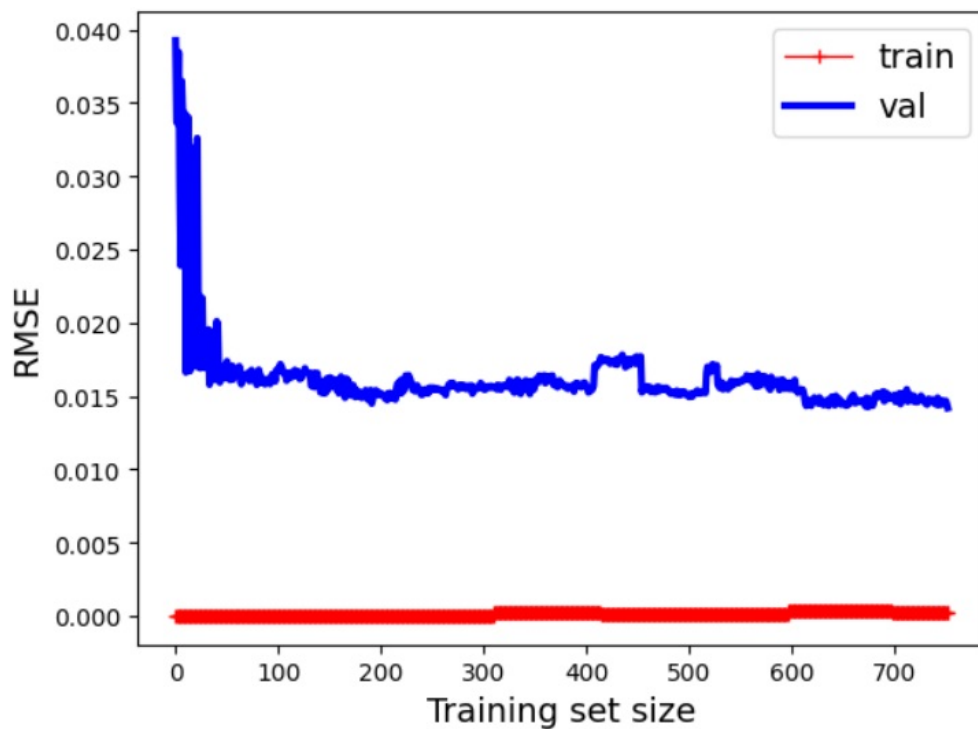
Test set:

Predictions: [0.01284689 0.02296651 0.03349282 0.0430622 0.01497608]
Labels: [0.008133971291866028, 0.03110047846889952, 0.02631578947368421, 0.029665071770334926, 0.023669856459330143]
RMSE: 0.01683212105928284

Cross Validation:

Scores: [0.02061468 0.01816489 0.01955152 0.01467669 0.01532377 0.01454708
0.0163643 0.01620629 0.01728478 0.01582199]
Average: 0.016855988567019
Standard deviation: 0.0019352482737005744

Кривая обучения:



По сравнению со стохастическим градиентным спуском ошибка уменьшилась. Из графика очевидно что модель переобучается, чтобы исправить это можно попробовать метод регуляризации (добавление некоторых дополнительных ограничений к условию). Для нахождения гиперпараметров воспользуемся методом решётчатого поиска. Воспользуемся методом и посмотрим на результаты:

Train set:

Predictions: [0.01284689 0.02296651 0.03349282 0.0430622 0.01497608]
 Labels: [0.016267942583732056, 0.02727272727272727, 0.022966507177033493, 0.015789473684210527, 0.04066985645933015]
 RMSE: 0.011641745037759355

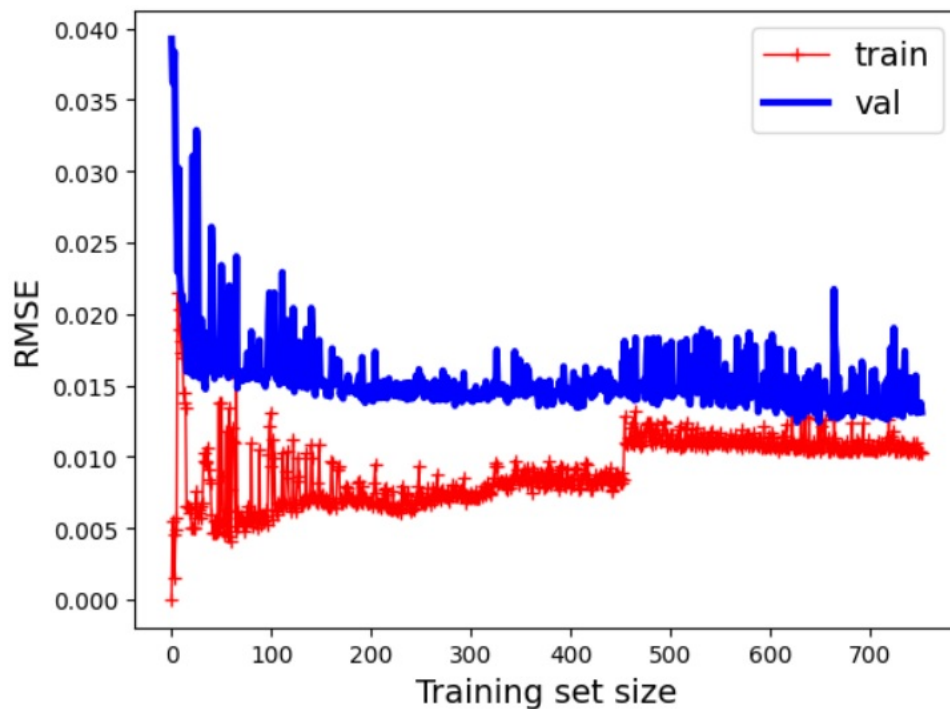
Test set:

Predictions: [0.01211692 0.02668702 0.03508695 0.03508695 0.02668702]
 Labels: [0.008133971291866028, 0.03110047846889952, 0.02631578947368421, 0.029665071770334926, 0.023669856459330143]
 RMSE: 0.012570998907705137

Cross Validation:

Scores: [0.01616952 0.01435054 0.02530278 0.01055466 0.01205509 0.01108742
 0.0135646 0.02019672 0.01473048 0.01189002]
 Average: 0.014990182704002323
 Standard deviation: 0.004364500753856351

Кривая обучения:



Проблема переобучения была решена и ошибка уменьшилась, но время работы алгоритма сильно возросло.

3.4. Случайный лес

Алгоритм случайного леса — универсальный алгоритм машинного обучения, суть которого состоит в использовании ансамбля решающих деревьев. Само по себе решающее дерево предоставляет крайне невысокое качество классификации, но из-за большого их количества результат значительно улучшается. Также это один из немногих алгоритмов, который можно использовать в абсолютном большинстве задач.

Результаты алгоритма:

Train set:

Predictions: [0.0202512 0.02642861 0.02139096 0.01696014 0.04041333]
 Labels: [0.016267942583732056, 0.02727272727272727, 0.022966507177033493, 0.015789473684210527, 0.04066985645933015]
 RMSE: 0.005139270980971324

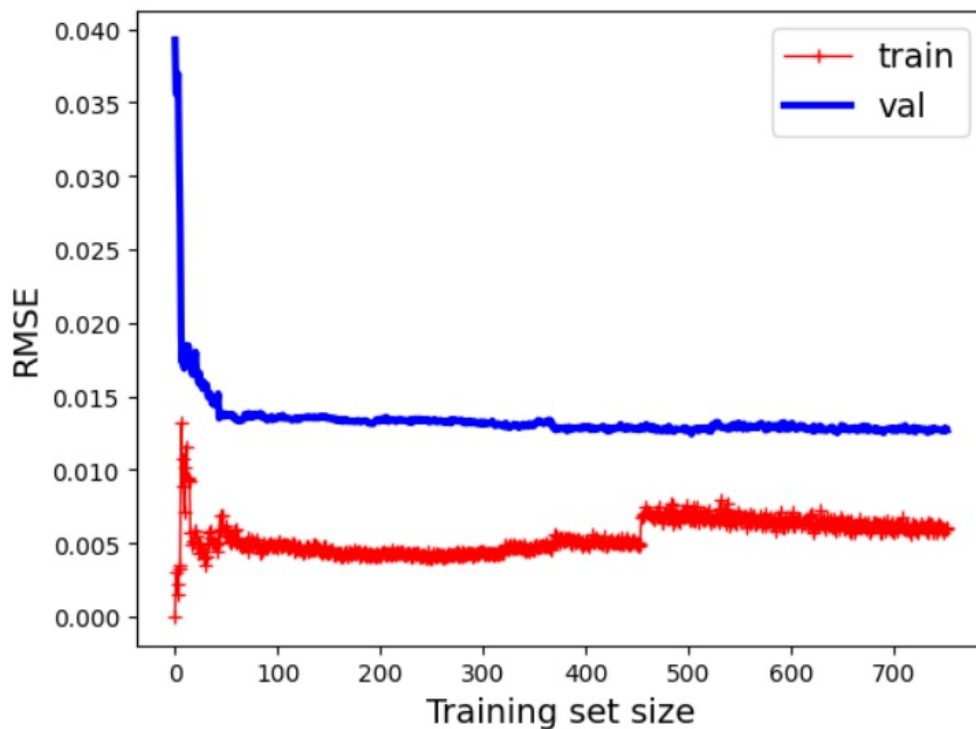
Test set:

Predictions: [0.01574952 0.02367167 0.03019833 0.03468096 0.01958799]
 Labels: [0.008133971291866028, 0.03110047846889952, 0.02631578947368421, 0.029665071770334926, 0.023669856459330143]
 RMSE: 0.011822102194708115

Cross Validation:

Scores: [0.02172502 0.01958373 0.03038721 0.01971744 0.01656162 0.02124272
 0.02189084 0.03750721 0.0236596 0.01967083]
 Average: 0.023194622875641467
 Standard deviation: 0.005882280239082831

Кривая обучения:



Ошибка увеличилась и модель переобучена.

3.5. Метод опорных векторов

Основная идея метода заключается в построении гиперплоскости, разделяющей объекты выборки оптимальным способом. Алгоритм работает в предположении, что чем больше расстояние (зазор) между разделяющей гиперплоскостью и объектами разделяемых классов, тем меньше будет средняя ошибка классификатора.

Результаты алгоритма:

Train set:

Predictions: [0.09146856 0.08690069 0.09512189 0.10567542 0.09905224]

Labels: [0.016267942583732056, 0.02727272727272727, 0.022966507177033493, 0.015789473684210527, 0.04066985645933015]

RMSE: 0.0665755507756366

Test set:

Predictions: [0.05073157 0.10348771 0.10424644 0.09581337 0.09686389]

Labels: [0.008133971291866028, 0.03110047846889952, 0.02631578947368421, 0.029665071770334926, 0.023669856459330143]

RMSE: 0.06699974995997099

Cross Validation:

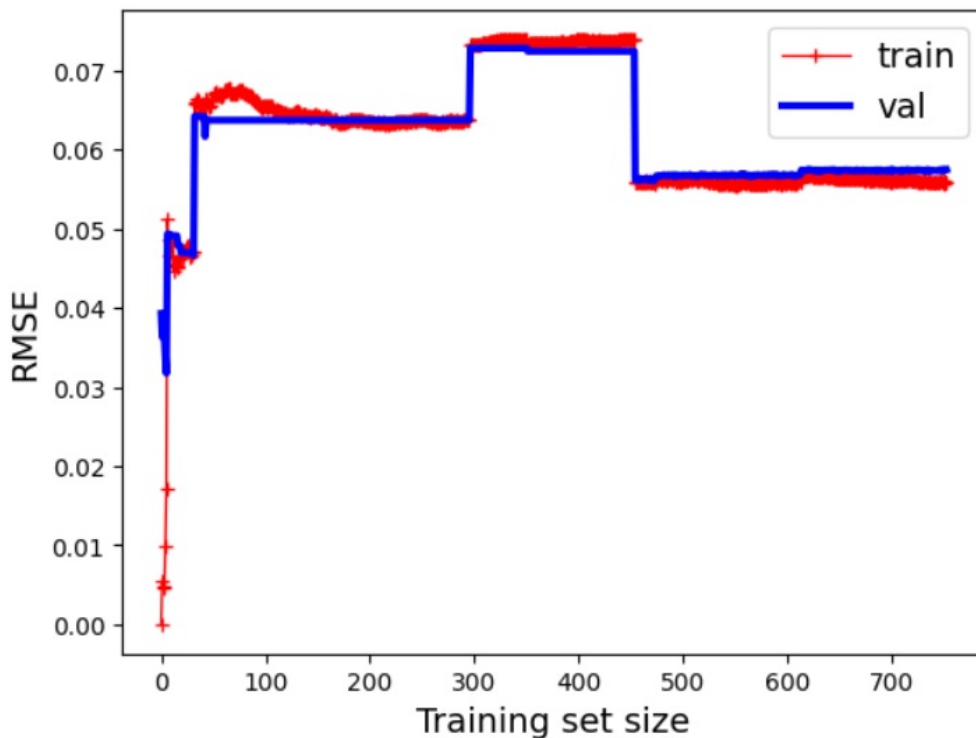
Scores: [0.06652354 0.067061 0.06460726 0.06669857 0.06592556 0.06732698

0.06677296 0.06473226 0.06479632 0.07139601]

Average: 0.0665840449723376

Standard deviation: 0.0018695805129602637

Кривая обучения:



Графики сходятся, но ошибка слишком велика - модель недообучена.

3.6. Гребневая регрессия

Гребневая регрессия является регуляризированной версией линейной регрессии: к функции издержек добавляется член регуляризации $\alpha \sum_{i=1}^n \theta_i^2$. Это заставляет алгоритм обучения не только приспосабливаться к данным, но также удерживать веса модели насколько возможно небольшими.

Функция издержек для гребневой регрессии

$$J(\theta) = \text{MSE}(\theta) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

Как и в случае линейной регрессии, мы можем производить гребневую регрессию, либо вычисляя уравнение в аналитическом виде, либо выполняя градиентный спуск. Доводы за и против одинаковы.

Решение в аналитическом виде для гребневой регрессии

$$\theta_{best} = (X^T * X + \alpha E)^{-1} * X^T * y$$

Результаты алгоритма:

Train set:

Predictions: [0.01928605 0.02756182 0.02327609 0.02749393 0.04233998]
Labels: [0.016267942583732056, 0.02727272727272727, 0.022966507177033493, 0.015789473684210527, 0.04066985645933015]
RMSE: 0.014618959024951205

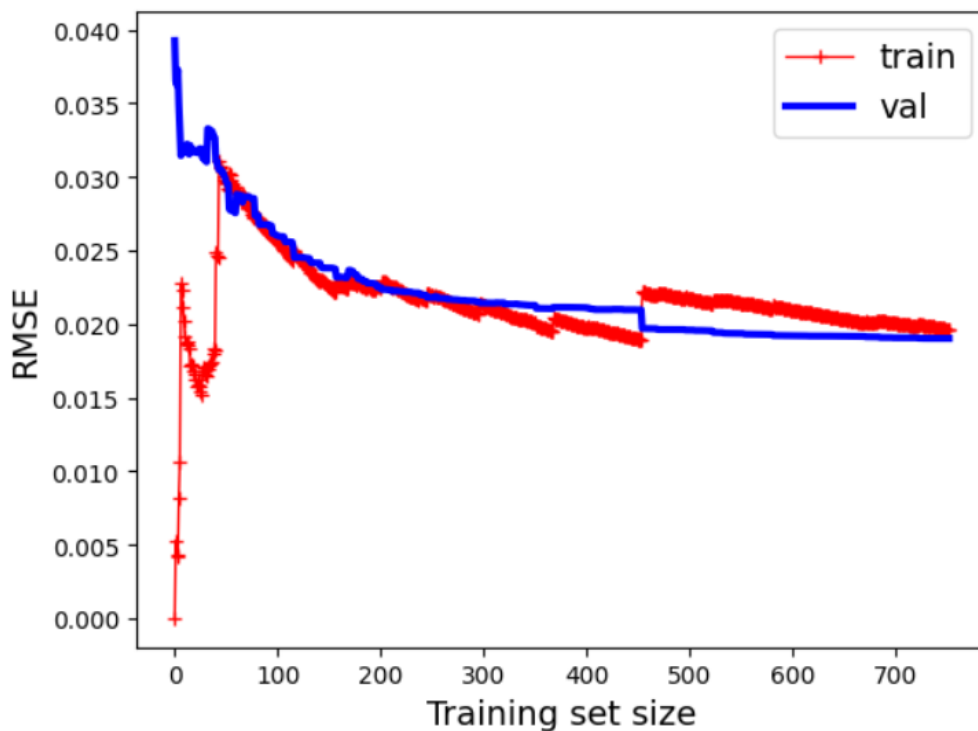
Test set:

Predictions: [0.01758507 0.02943711 0.03170689 0.0334561 0.0279789]
Labels: [0.008133971291866028, 0.03110047846889952, 0.02631578947368421, 0.029665071770334926, 0.023669856459330143]
RMSE: 0.01379716752502853

Cross Validation:

Scores: [0.01511193 0.01451884 0.01362546 0.01378029 0.01225427 0.0137423
0.01623897 0.01870156 0.01726153 0.01342527]
Average: 0.014866039911578467
Standard deviation: 0.0018813138129261534

Кривая обучения:



Ошибка уменьшилась и графики сходятся, неплохая модель.

4. Заключение

В ходе работы были использованы разные подходы к исследованию задачи регрессии: классическим подход, метод опорных векторов, дерево решений, случайный лес,

регрессия с голосованием, полиномиальная регрессия со стохастическим градиентным спуском. Линейная регрессия дала неплохой результат, применение более мощных моделей не помогло улучшить результаты, видимо данные слишком просты.

5. Список источников

Орельен Жерон, «Прикладное машинное обучение с помощью Scikit-Learn и TensorFlow». Диалектика, 2018

<https://ru.wikipedia.org/>

<https://www.kaggle.com/>

6. Приложение

```
import os
import pandas as pd
import numpy as np
from math import floor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import MinMaxScaler
from pandas.plotting import scatter_matrix
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import PolynomialFeatures
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import AdaBoostRegressor
from sklearn.svm import SVR
from sklearn.linear_model import Ridge
```

```

#uploading data
HOUSING_PATH = os.path.join("datasets")
def load_housing_data(housing_path=HOUSING_PATH):
    csv_path = os.path.join(housing_path, "Pune house data.csv")
    return pd.read_csv(csv_path)
table = load_housing_data()
table

#remove unnecessary attributes
table = table.drop("availability", axis=1)
table = table.drop("society", axis=1)
table

#check null values
table.isnull().sum()
#discard null values
table['balcony'] = table['balcony'].fillna('0')
table = table.dropna()
table.isnull().sum()
table.shape

table['num_size'] = table['size'].apply(lambda x: int(x.split('_')[0]))
table = table.drop("size", axis=1)
def sqft_fix(x):
    t = x.split('-')
    if len(t) == 2:
        return (float(t[0]) + float(t[1]))/2
    try:
        return float(x)
    except:
        return None
table["total_sqft"] = table["total_sqft"].apply(sqft_fix)
table

```

```

#check null values
table.isnull().sum()

table = table.dropna()
table.isnull().sum()
table['price_per_sqft'] = table['price'] * 100000 / table['total_sqft']
table = table[~(table.total_sqft/table.num_size<300)]
table.shape

def remove_pps_outliers(df):
    res = pd.DataFrame()
    for key, subdf in df.groupby('site_location'):
        m = np.mean(subdf.price_per_sqft)
        st = np.std(subdf.price_per_sqft)
        reduced_df = subdf[(subdf.price_per_sqft>(m-st))
                            & (subdf.price_per_sqft<=(m+st))]
        res = pd.concat([res, reduced_df],
                        ignore_index = True)

    return res

table = remove_pps_outliers(table)
table.shape

def remove_num_size_outliers(df):
    exclude_indices = np.array([])
    for location, location_df in df.groupby('site_location'):
        num_size_stats = {}
        for num_size, num_size_df in location_df.groupby(
            'num_size'):
            num_size_stats[num_size] = {
                'mean' : np.mean(
                    num_size_df.price_per_sqft),
                'std' : np.std(
                    num_size_df.price_per_sqft),

```

```

        'count' : num_size_df.shape[0]
    }
    for num_size, num_size_df in location_df.groupby(
        'num_size'):
        stats = num_size_stats.get(num_size-1)
        if stats and stats['count']>5:
            exclude_indices = np.append(
                exclude_indices,
                num_size_df[num_size
                    _df.price_per_sqft<(
                        stats['mean'])].index.values)
    return df.drop(exclude_indices, axis='index')
table = remove_num_size_outliers(table)
table.shape
table = table.drop("price_per_sqft", axis=1)

dummy_cols = pd.get_dummies(table.area_type).drop(
    'Super_built-up_Area', axis='columns')
table = pd.concat([table, dummy_cols], axis='columns')
table = table.drop("area_type", axis=1)
table["site_location"].value_counts()
table["site_location"] = table["site_location"].astype('category')
table.dtypes
table["site_location"] = table["site_location"].cat.codes
table

table.corr()["price"].sort_values(ascending=False)

#min-max scaling
table_col = table.columns.values
MMS = MinMaxScaler(copy=False).fit(table)
table = MMS.transform(table)
table = pd.DataFrame(table, columns=table_col)
table

```

```

#graphics
scatter_matrix(table[["price", "total_sqft", "bath", "num_size"]],
                figsize=(12, 8))

#split train test
table_labels = table['price'].copy()
table_ready = table.drop('price', axis='columns')
train_set, test_set, train_labels, test_labels =
    train_test_split(table_ready, table_labels, test_size = 0.2,
                    random_state = 42)
train_set.head()

#learning curves
def plot_learning_curves(model):
    X_train = train_set.values
    X_val = test_set.values
    y_train = train_labels.values
    y_val = test_labels.values
    train_errors, val_errors = [], []
    for m in range(1, len(X_train) // 8):
        model.fit(X_train[:m], y_train[:m])
        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)
        train_errors.append(mean_squared_error(
            y_train_predict, y_train[:m]))
        val_errors.append(mean_squared_error(
            y_val_predict, y_val))
    plt.plot(np.sqrt(train_errors), "r+",
             linewidth=1, label="train")
    plt.plot(np.sqrt(val_errors), "b-", linewidth=3, label="val")
    plt.legend(loc="upper right", fontsize=14)
    plt.xlabel("Training set size", fontsize=14)
    plt.ylabel("RMSE", fontsize=14)

```

```

def display_scores(model, t_set):
    scores = cross_val_score(model, t_set, train_labels,
                              scoring="neg_mean_squared_error", cv=10)
    rmse_scores = np.sqrt(-scores)
    print("Scores:", rmse_scores)
    print("Average:", rmse_scores.mean())
    print("Standard deviation:", rmse_scores.std(), "\n")

#linear regression
lin_reg = LinearRegression()
lin_reg.fit(train_set, train_labels)
print("Train set:\n")
lin_reg_res = lin_reg.predict(train_set)
print("Predictions:", lin_reg_res[:5])
print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, lin_reg_res)), '\n')
print("Test set:\n")
lin_reg_res = lin_reg.predict(test_set)
print("Predictions:", lin_reg_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    test_labels, lin_reg_res)), '\n')
print("Cross Validation:\n")
display_scores(lin_reg, train_set)
print("Learning curves:")
plot_learning_curves(lin_reg)

sgd_reg = SGDRegressor(max_iter=250, penalty=None, eta0=0.1)
sgd_reg.fit(train_set_poly, train_labels)
print("Train set:\n")
sgd_reg_res = sgd_reg.predict(train_set_poly)
print("Predictions:", sgd_reg_res[:5])

```

```

print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, sgd_reg_res)), '\n')
print("Test set:\n")
sgd_reg_res = sgd_reg.predict(test_set_poly)
print("Predictions:", sgd_reg_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    test_labels, sgd_reg_res)), '\n')
print("Cross Validation:\n")
display_scores(sgd_reg, train_set_poly)
print("Learning curves:")
plot_learning_curves(sgd_reg)

forest_reg = RandomForestRegressor()
forest_reg.fit(train_set, train_labels)
print("Train set:\n")
forest_reg_res = forest_reg.predict(train_set)
print("Predictions:", forest_reg_res[:5])
print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, forest_reg_res)), '\n')
print("Test set:\n")
forest_reg_res = forest_reg.predict(test_set)
print("Predictions:", forest_reg_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    test_labels, forest_reg_res)), '\n')
print("Cross Validation:\n")
display_scores(sgd_reg, train_set)
print("Learning curves:")
plot_learning_curves(forest_reg)

dtr = DecisionTreeRegressor()

```

```

dtr.fit(train_set, train_labels)
print("Train set:\n")
dtr_res = dtr.predict(train_set)
print("Predictions:", dtr_res[:5])
print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, dtr_res)), '\n')
print("Test set:\n")
dtr_res = dtr.predict(test_set)
print("Predictions:", dtr_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    test_labels, dtr_res)), '\n')
print("Cross Validation:\n")
display_scores(dtr, train_set)
print("Learning curves:")
plot_learning_curves(dtr)

hyperparameters = [{ 'max_depth': [20, 21, 22, 23],
                      'min_samples_leaf': [2, 3, 4, 5],
                      'min_samples_split': [2, 4, 6, 8],
                      'max_features': [6, 7, 8, 9],
                      'max_leaf_nodes': [23, 24, 25, 26]}]
dtr = DecisionTreeRegressor()
dtr_hp = GridSearchCV(dtr, hyperparameters, cv=10,
                      scoring='neg_mean_squared_error')
dtr_hp.fit(train_set, train_labels)
print("Train set:\n")
dtr_hp_res = dtr_hp.best_estimator_.predict(train_set)
print("Predictions:", dtr_res[:5])
print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, dtr_hp_res)), '\n')
print("Test set:\n")

```



```

dtr_hp_res = dtr_hp.best_estimator_.predict(test_set)
print("Predictions:", dtr_hp_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    (test_labels, dtr_hp_res))), '\n')
print("Cross Validation:\n")
display_scores(dtr_hp.best_estimator_, train_set)
print("Learning curves:")
plot_learning_curves(dtr_hp.best_estimator_)

abr = AdaBoostRegressor(dtr_hp.best_estimator_,
    n_estimators=150, learning_rate=0.5, loss='square')
abr.fit(train_set, train_labels)
print("Train set:\n")
abr_res = abr.predict(train_set)
print("Predictions:", abr_res[:5])
print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, abr_res))), '\n')
print("Test set:\n")
abr_res = abr.predict(test_set)
print("Predictions:", abr_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    (test_labels, abr_res))), '\n')
print("Cross Validation:\n")
display_scores(abr, train_set)
print("Learning curves:")
plot_learning_curves(abr)

svr = SVR(kernel="poly", degree=2, C=150, epsilon=0.1)
svr.fit(train_set, train_labels)
print("Train set:\n")
svr_res = svr.predict(train_set)

```

```

print("Predictions:", svr_res[:5])
print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, svr_res)), '\n')
print("Test set:\n")
svr_res = svr.predict(test_set)
print("Predictions:", svr_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    test_labels, svr_res)), '\n')
print("Cross Validation:\n")
display_scores(svr, train_set)
print("Learning curves:")
plot_learning_curves(svr)

ridge_reg = Ridge(alpha=1, solver = "cholesky")
ridge_reg.fit(train_set, train_labels)
print("Train set:\n")
ridge_reg_res = ridge_reg.predict(train_set)
print("Predictions:", ridge_reg_res[:5])
print("Labels:", list(train_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    train_labels, ridge_reg_res)), '\n')
print("Test set:\n")
ridge_reg_res = ridge_reg.predict(test_set)
print("Predictions:", ridge_reg_res[:5])
print("Labels:", list(test_labels.values)[:5])
print("RMSE:", np.sqrt(mean_squared_error(
    test_labels, ridge_reg_res)), '\n')
print("Cross Validation:\n")
display_scores(ridge_reg, train_set)
print("Learning curves:")
plot_learning_curves(ridge_reg)

```